

Análisis de Rendimiento de Estudiantes

Comprensión de Listas

Dirigido: Jose Serra Reina

Fecha: 22/03/26

Análisis de Requerimientos de Misión.

¿Qué es?

El Análisis de Requerimientos de Misión (MRA) en el ámbito de la programación y el desarrollo de sistemas complejos es el proceso de traducir los objetivos estratégicos de alto nivel (el "qué" y "para qué" de la operación global) en especificaciones técnicas y funcionales que el software debe cumplir para garantizar el éxito.

A diferencia del análisis de requerimientos convencional, que suele enfocarse en las necesidades del usuario final o la interfaz, el MRA se centra en la supervivencia de la misión y el comportamiento del sistema en entornos operativos críticos.

¿Estructura de análisis y definición?

Enunciado del Proyecto

Desarrolla una serie de operaciones en Python para **procesar y analizar las calificaciones obtenidas** por un grupo de estudiantes en una asignatura. El objetivo es transformar una lista inicial de datos brutos en estructuras más útiles para el análisis, como obtener las calificaciones únicas, calcular promedios, y filtrar a los estudiantes aprobados, todo ello utilizando **comprensiones**.

Pasos Clave y Uso de Comprensiones

1. Lista Inicial (Datos de Origen):

Empieza con una lista de tuplas donde cada tupla representa a un estudiante: (**Nombre**, **Calificación**). generar los datos de los estudiantes desde un archivo externo aleatorio y fusionar todo...tratar de que ambos datos como las notas y los nombres se asignen en la lista que tengo si es posible y mostrar el resultado.

Python

```
datos_estudiantes = [  
    ("Ana", 85),  
    ("Luis", 60),  
    ("Marta", 92),
```

```
("Pedro", 85),
("Ana", 78), # Ana tiene dos registros
("Javier", 60),
("Sara", 92),
("Luis", 45) # Luis tiene dos registros
]
```

2. Uso de Comprensiones:

A. 📖 Comprensión de Lista (List Comprehension)

Objetivo: Obtener una lista con el nombre de **todos los estudiantes que han aprobado** (calificación ≥ 70).

- **Sintaxis:** `[expresión for elemento in iterable if condición]`
- **Tarea:** Generar una lista de *strings* con solo el nombre de los aprobados.

B. 📊 Comprensión de Diccionario (Dictionary Comprehension)//Parte 3:

Objetivo: Crear un diccionario que asigne a cada estudiante su **calificación más alta** (eliminando duplicados).

- **Sintaxis:** `{clave: valor for elemento in iterable}`
- **Tarea:** La clave es el nombre, y el valor es su nota máxima registrada.
 - (Sugerencia: Tendrás que iterar sobre los datos y usar una lógica de maximización.)

C. 📦 Comprensión de Conjunto (Set Comprehension):

Objetivo: Determinar el conjunto de **calificaciones únicas** obtenidas en el grupo.

- **Sintaxis:** `{expresión for elemento in iterable}`
- **Tarea:** Generar un *set* de números enteros que muestre qué calificaciones se obtuvieron al menos una vez (ej: `{85, 60, 92, 78, 45}`).

D. ⚖️ Expresión Generadora (Generator Expression)

Objetivo: Calcular la **suma total de todas las calificaciones** para luego obtener el promedio general, sin almacenar la lista intermedia de calificaciones en memoria.

- **Sintaxis:** `(expresión for elemento in iterable)`
- **Tarea:** Crear un generador que devuelva cada calificación de la lista original. Luego, usa la función `sum()` con el generador para obtener el total.

Este proyecto te obliga a utilizar las cuatro estructuras en un contexto práctico de manipulación de datos

¿Conceptos previos indispensables?

→ Parte 1: Preparación y Carga de Datos

Para esta etapa, necesitas entender cómo Python interactúa con el sistema operativo y cómo estructurar datos básicos.

- Manejo de Archivos (I/O): Es la capacidad de abrir, leer y cerrar archivos externos (como `.txt` o `.csv`). Se recomienda el uso del gestor de contexto `with open()`, que asegura que el archivo se cierre correctamente.
- Módulo `random`: Librería estándar para generar datos aleatorios (nombres o notas), útil para simular escenarios de prueba.
- Estructuras de Datos (Tuplas y Listas): Las Tuplas son secuencias inmutables (no cambian) usadas para representar registros fijos como (`Nombre`, `Nota`). Las Listas son colecciones mutables que actúan como el contenedor principal de tu dataset.
- Zip y Unpacking: Técnica para combinar dos iterables (una lista de nombres y una de notas) elemento a elemento.

→ Parte 2: Comprensión de Lista (Filtrado)

Aquí aplicas lógica condicional dentro de una estructura compacta.

- List Comprehension: Una forma sintáctica de crear una nueva lista a partir de una existente. Es más eficiente y legible que un bucle `for` tradicional.
- Lógica Booleana (Filtros): El uso de operadores de comparación (`>`, `<`, `==`) dentro de la cláusula `if` de la comprensión para decidir qué elementos incluir.

→ Parte 3: Comprensión de Diccionario (Lógica de Máximos)

Esta es la parte más técnica, ya que implica transformar y comparar datos.

- Dictionary Comprehension: Sintaxis para construir diccionarios de forma dinámica: `{llave: valor for elemento in iterable}`.
- Función `sorted()` o `max()`: Para manejar duplicados (como "Ana" que aparece dos veces), necesitas saber ordenar los datos previamente o usar funciones de agregación para que el diccionario solo conserve la calificación más alta.
- Acceso por Llave: Entender que en un diccionario las llaves son únicas; si asignas un valor a una llave que ya existe, este se sobrescribe.

→ Parte 4: Comprensión de Conjunto (Unicidad)

Se enfoca en la teoría de conjuntos aplicada a la programación.

- Set Comprehension: Similar a la de lista, pero utiliza llaves `{}`. Su propiedad principal es que elimina automáticamente los duplicados.
- Hashing: Concepto teórico (opcional pero útil) de por qué los sets son tan rápidos para encontrar valores únicos y por qué sus elementos deben ser inmutables (como números o strings).

→ Parte 5: Expresión Generadora (Eficiencia de Memoria)

Aquí pasamos de la manipulación de datos a la optimización de recursos.

- Iteradores vs. Iterables: Un iterable es una colección (como una lista); un Iterador (Generador) es un objeto que produce valores uno a uno solo cuando se le piden.
- Lazy Evaluation (Evaluación Perezosa): Es la estrategia de no procesar los datos hasta el momento exacto en que se necesitan. Esto ahorra memoria RAM al no cargar miles de datos simultáneamente.
- Funciones de Reducción (`sum`, `len`): Funciones que toman un iterable o generador y devuelven un único valor (el promedio es el resultado de `sum() / cantidad`).

→ Parte 6 y 7: Prueba y Despliegue

Conocimientos de flujo de trabajo profesional.

- Modularización: Organizar el código en funciones o secciones claras.
- Control de Versiones (Git/GitHub): Conocimiento básico para crear un repositorio, hacer un *commit* y subir tu código (el "Push") para que sea visible en la red.
- Markdown: Lenguaje de marcado sencillo para escribir el archivo `README.md` que explicará tu proyecto en GitHub.

→ Resumen Técnico de Apoyo

Dato Clave: Las comprensiones no solo son "azúcar sintáctico" (hacer el código más bonito), sino que en Python suelen ejecutarse a una velocidad cercana al lenguaje C interno, siendo más rápidas que los bucles `for` manuales.

¿Por qué lo son y de qué manera se integraron(beneficio) ?

→ Parte 1: Preparación y Carga de Datos

- **Herramientas:** `with open()`, `random`, `zip()`.
- **El porqué:** Porque necesitas convertir datos externos e inconexos en una estructura de datos coherente.
- **Beneficio de la integración:** Al usar **Manejo de Archivos**, evitas el ingreso manual; con `random`, generas volumen de prueba; y con `zip()`, aseguras la **integridad referencial** (que el nombre coincida con la nota).

→ Parte 2: Comprensión de Lista (Filtrado)

- **Herramientas:** `List Comprehension + if condition`.
- **El porqué:** Es la forma más eficiente en Python de crear subconjuntos de datos.
- **Beneficio de la integración: Velocidad de procesamiento.** Al integrar el filtro directamente en la creación de la lista, el intérprete de Python optimiza el ciclo, lo que resulta en un código más limpio y rápido que un bucle tradicional.

→ Parte 3: Comprensión de Diccionario (Lógica de Máximos)

- **Herramientas:** `Dict Comprehension + sorted()`.
- **El porqué:** Porque un diccionario es la estructura óptima para búsquedas por clave (Nombre), pero requiere una técnica para decidir qué valor (Nota) conservar ante duplicados.
- **Beneficio de la integración: Precisión de datos.** Al integrar el ordenamiento (`sorted`) antes de la comprensión, obligas al diccionario a "quedarse" con la mejor versión de la información, resolviendo el problema de los duplicados de forma elegante.

→ Parte 4: Comprensión de Conjunto (Unicidad)

- **Herramientas:** `Set Comprehension {}`.
- **El porqué:** Los conjuntos están programados para rechazar duplicados por naturaleza algorítmica.

- **Beneficio de la integración: Reducción de entropía.** Obtienes el espectro de calificaciones únicas sin escribir lógica extra de comparación. Es una operación de **costo computacional mínimo**.

→ Parte 5: Expresión Generadora (Eficiencia de Memoria)

- **Herramientas:** `Generator Expression ()`.
- **El porqué:** Porque crear una lista de promedios en memoria es innecesario si solo quieres el resultado final del cálculo.
- **Beneficio de la integración: Escalabilidad.** Integrar generadores permite que tu programa maneje millones de registros sin que el consumo de memoria suba. Es la diferencia entre un código de estudiante y uno de **Ingeniero en Computación**.

→ Parte 6: Prueba

- **Herramientas:** `Modularización` y `Prints` formateados.
- **El porqué:** Para verificar que la "misión" del software se cumple.
- **Beneficio de la integración: Confiabilidad.** Aseguras que cada una de las transformaciones de datos anteriores funcionó según lo previsto.

→ Parte 7: Subirlo a la red

- **Herramientas:** `Git`, `GitHub`, `Markdown`.
- **El porqué:** Para documentar el proceso y demostrar profesionalismo.
- **Beneficio de la integración: Legado y Portafolio.** Permite que tu trabajo sea auditable y te posiciona como un desarrollador que domina la optimización de datos.

Interfaz.

¿Qué tipo de herramienta es ?

CustomTkinter es una biblioteca de Python basada en Tkinter que sirve para modernizar radicalmente el aspecto de las interfaces gráficas (GUI).

Si alguna vez has usado el Tkinter estándar, habrás notado que las ventanas parecen sacadas de Windows 95 o XP. CustomTkinter soluciona esto envolviendo los componentes clásicos con un diseño contemporáneo, esquinas redondeadas y soporte nativo para temas.

¿Quien la desarrolló ?

Fue desarrollada principalmente por **Tom Schimansky**.

Es un desarrollador alemán que lanzó el proyecto como una extensión para modernizar **Tkinter**, ya que las interfaces estándar de Python se sentían muy anticuadas. Aunque Schimansky es el autor principal y quien mantiene el repositorio oficial, el proyecto es de código abierto, por lo que ha recibido contribuciones de otros desarrolladores de la comunidad en GitHub.

¿A qué Librería pertenece ?

1. Librerías Complementarias Esenciales

CustomTkinter por sí solo es el "esqueleto", pero para que el expediente funcione, necesitas estas tres:

- **Pillow (PIL): Imprescindible.** La necesitarás para cargar las fotos de los estudiantes, redimensionarlas y que queden perfectas dentro de los marcos del expediente.
 - **CTkMessageBox:** Es una extensión popular para CustomTkinter que te permite mostrar alertas estilo "Acceso Concedido", "Alumno Reprobado" o errores de carga de archivo con una estética moderna.
 - **Matplotlib:** Para integrar los gráficos de rendimiento (el promedio que calculas en la Parte 5) directamente en una ventana de la interfaz.
-

2. Conceptos Clave de CustomTkinter (CTK)

Antes de empezar, dale una revisada a estos componentes; son los que darán vida al colegio:

A. CTkScrollableFrame (La lista de alumnos)

En lugar de una lista estática, usa este contenedor para mostrar a todos los estudiantes de tu archivo. Permite hacer scroll, lo cual es vital si tu archivo externo tiene muchos nombres.

B. CTkTabview (Organización del Expediente)

*Te permite crear "pestañas". Podrías tener una pestaña para **"Datos Personales"**, otra para **"Calificaciones"** (donde muestras tu diccionario de máximos) y otra para **"Estadísticas"** (donde muestras el set de notas únicas).*

C. CTkImage (El manejo de fotos)

Es el puente entre Pillow y CustomTkinter. Es lo que te permitirá mostrar la imagen del estudiante con alta calidad y escalado dinámico.

3. Arquitectura: Programación Orientada a Objetos (POO)

*Aunque puedes hacer cosas simples de forma procedimental, para este proyecto te recomiendo aprender la estructura de **Clases** en CTK.*

- **¿Por qué?:** Te permite separar la lógica de tu "motor" de búsqueda del diseño de la ventana.
 - **Beneficio:** Podrás crear una clase `ExpedienteView` que se actualice automáticamente cada vez que selecciones un nombre diferente de tu lista de tuplas.
-

4. El "Layout Manager": `.grid()` vs `.pack()`

En ingeniería de interfaces, el orden es todo.

- Aprende a usar `.grid()`. Te permite dividir tu ventana en filas y columnas (como una hoja de Excel). Es mucho más preciso para colocar la foto a la izquierda, los datos al centro y las gráficas a la derecha.
-

Hoja de Ruta Sugerida

1. **Instalación:** `pip install customtkinter pillow`.
2. **Prueba de Concepto:** Crea una ventana simple con un botón que, al presionarlo, imprima tu **Lista de Aprobados** (de la Parte 2) en la terminal.
3. **Integración de Imagen:** Intenta mostrar una imagen cualquiera en un `CTkLabel`. Si logras esto, ya tienes el 50% de la interfaz ganada.

Tip Pro: Revisa la documentación oficial o ejemplos de "Complex Example" en el repositorio de CustomTkinter. Verás cómo usan los `Frames` para dividir la pantalla en secciones limpias.

¿Función en el proyecto?

La interfaz no es un adorno; es el **operador de misión** que permite que un usuario (profesor o administrativo) interactúe con tus algoritmos sin ver el código. Aquí tienes la función de la interfaz desglosada por las etapas de tu proyecto:

1. Etapa de Entrada (Parte 1): El Portal de Datos

- **Función:** Actúa como el **mecanismo de ingesta**. En lugar de modificar el código para cambiar de archivo, la interfaz ofrece un botón de "Cargar Archivo" o un buscador.
- **Beneficio:** Humaniza el acceso a los datos. Permite que el sistema sea usado por alguien que no sabe programar.

2. Etapa de Visualización de Filtros (Parte 2): El Monitor de Estatus

- **Función:** Es el **traductor visual** de tu List Comprehension. En la terminal verías una lista de strings; en la interfaz, esto se convierte en una lista de carnets escolares con colores.
- **Beneficio:** Permite identificar de un vistazo quién aprobó y quién no, usando el impacto visual (Verde/Rojo) en lugar de lectura de texto plano.

3. Etapa de Gestión de Identidad (Parte 3): El Expediente Central

- **Función:** Es el **unificador de información**. Aquí la interfaz toma el diccionario de notas máximas y lo presenta como una boleta de calificaciones formal.
- **Beneficio:** Resuelve la complejidad de los duplicados para el usuario. El usuario solo ve el "Resultado Final Optimizado" de cada estudiante, ocultando el caos de los datos brutos iniciales.

4. Etapa de Auditoría (Parte 4): La Vista de Calidad

- **Función:** Mostrar el **espectro de datos únicos** (Set). La interfaz puede presentar esto como una leyenda o etiquetas de "Calificaciones Detectadas en este Periodo".
- **Beneficio:** Sirve para verificar la variedad de las notas sin repeticiones, dando una idea rápida de qué tan diverso fue el rendimiento del grupo.

5. Etapa de Inteligencia de Negocios (Parte 5): El Dashboard de Eficiencia

- **Función:** Es la **ventana de rendimiento**. Toma el resultado de tu expresión generadora y lo proyecta en un gráfico de barras o una métrica destacada.
- **Beneficio:** Convierte un número frío (el promedio) en una comparación visual. Es donde la interfaz demuestra la eficiencia de memoria, mostrando resultados en tiempo real sin congelar la aplicación.

Paradigma

¿Nombre ?

Paradigma: Estructurado/Imperativo .

¿En que consiste ?

El "Cómo": Se centra en describir paso a paso las instrucciones que la computadora debe seguir para resolver un problema.

Estado Mutable: Se basa en el uso de variables que cambian su valor a lo largo de la ejecución.

Control de Flujo: Utiliza estructuras lógicas básicas:

1. Secuencias (paso a paso).
2. Condicionales (`if/else`).
3. Bucles (`for/while`).

Modularidad: Divide el código en funciones o procedimientos para evitar el desorden y reutilizar lógica.

estructura en el proyecto ? ¿pseudocodigo//estructura//diagrama?

PROYECTO_RENDIMIENTO/

```
├── core/                                # Lógica Pura (Paradigma Estructurado)
│   ├── __init__.py
│   ├── transformaciones.py            # Aquí viven las 4 comprensiones
│   └── generador_datos.py             # Lectura de archivos y randomización
├── interface/                          # Capa Visual (CustomTkinter)
│   ├── __init__.py
│   ├── ventana_ingreso.py            # Panel de control y tabla tiempo real
│   └── ventana_fbi.py                 # El "Dossier" con estética de seguridad
├── data/                               # Repositorio de Información
│   └── fuentes_externas.txt           # Tu base de nombres aleatorios
└── main.py                            # Orquestador Principal
```

Gestión de avances(Indicaciones del repositorio)

//Incluir si son necesarios los comentarios(versiones previas).

¿Estructura?

Logico:

Pas o	Rama	Mensaje de Commit (Buena Práctica)	Alcance del Avance
1	feature/engine-logic	feat: add IO module to read and merge random student data	Implementación de with open, random y zip para generar la lista inicial.
2	feature/engine-logic	feat: implement list comprehension for approved students	Filtro de aprobados (≥ 70) mediante list comprehension.
3	feature/engine-logic	feat: implement dict comprehension for maximum grades	Lógica de ordenamiento y agrupamiento para eliminar duplicados con la nota más alta.
4	feature/engine-logic	feat: implement set comprehension for unique grades	Extracción de calificaciones únicas sin duplicados.
5	feature/engine-logic	feat: implement generator expression for	Uso de evaluación perezosa (<i>lazy evaluation</i>) para

		memory-efficient average	calcular el promedio con sum() .
--	--	--------------------------	---

Gráfica:

Pas o	Rama	Mensaje de Commit (Buena Práctica)	Alcance del Avance
1	feature/gu i-interface	feat: initialize CustomTkinter window with grid layout	Ventana principal configurada con filas y columnas proporcionales.
2	feature/gu i-interface	feat: integrate CTkScrollableFrame for student roster	Contenedor dinámico lateral para listar los estudiantes cargados.
3	feature/gu i-interface	feat: implement CTkTabview for metrics and maximum grades	Pestañas de visualización segregada (Aprobados, Máximos, Estadísticas).

4	feature/gui i-interface	feat: embed Matplotlib canvas for performance charts	Gráfico de barras integrado que lee los datos filtrados en tiempo real.
5	feature/gui i-interface	docs: complete README with operational architecture and run instructions	Documentación final en Markdown con diagramas de uso del sistema.

Estructura en Github:

Plaintext

```

analisis-estudiantes/
├── .gitignore          # Excluye __pycache__, entornos virtuales (.venv) y archivos
├── README.md           # Documentación del proyecto, requerimientos de misión y cómo
├── nombres.txt         # Archivo de texto fuente para la ingesta de nombres de estu
├── student_engine.py   # Módulo de backend (Fase 1: Algoritmos de Comprensión)
└── student_gui.py      # Módulo de frontend (Fase 2: Interfaz de Usuario de Alta Fi

```

Indicaciones(ejemplos de uso)

¿Requerimientos Iniciales?

¿Estructura?

¿Funcionalidad?

¿Capacidades Extendidas?

¿Alertas?